

Job Application Agent with Autonomous Research & Monitoring

Complete Project Blueprint — Architecture to Deployment

FULL STACK	AGENTIC AI	DEVOPS	PAYMENTS	100% FREE
------------	------------	--------	----------	-----------

Tech Stack

Next.js 14 | TypeScript | Prisma | PostgreSQL | Redis + BullMQ | Groq API | Playwright |
Razorpay | Docker | Nginx | Cloudflare

Timeline:	4-5 Months (part-time)
Daily Commitment:	2 hrs weekdays + 5 hrs weekends
Cost:	Completely Free to Build
Level:	Intermediate (Junior → Mid-Level)
Target Outcome:	Portfolio project + live SaaS + salary jump

Personalized learning blueprint | April 2026

Table of Contents

1	Project Overview & What You Will Build
2	Why This Project Stands Out
3	Complete Tech Stack Breakdown
4	System Architecture & Data Flow
5	Database Schema Design
6	Free vs Pro Feature Split
7	Phase 1 — Project Setup & Foundation (Week 1-2)
8	Phase 2 — Resume Upload & Parsing (Week 3-4)
9	Phase 3 — Job Scraping & Research Agent (Week 5-7)
10	Phase 4 — Dashboard & Analytics (Week 8-9)
11	Phase 5 — AI Chat & Resume Tailoring (Week 10-12)
12	Phase 6 — Weekly Monitoring Agent (Week 13-14)
13	Phase 7 — Razorpay Payments & Subscriptions (Week 15-16)
14	Phase 8 — Docker, Nginx & Deployment (Week 17-18)
15	API Endpoints Reference
16	Folder Structure
17	Environment Variables & Free Services Setup
18	DSA Concepts You Will Learn Naturally
19	Common Mistakes & How to Avoid Them
20	Interview Talking Points from This Project
21	Launch Checklist & Post-Build Strategy

1 Project Overview & What You Will Build

This project is an **AI-powered Job Application Agent** — a SaaS tool that helps job seekers apply smarter, not harder. It is not a generic resume builder and not a chat wrapper around ChatGPT. It is a full production-grade application with autonomous agents that research companies, tailor resumes, monitor application pipelines, and warn users before they waste applications on companies in trouble.

What the finished product does:

- **Resume Upload & Parsing:** Users upload their resume (PDF or DOCX). The system parses it into structured data — skills, experience, projects, education.
- **Job URL Ingestion:** Users paste any job posting URL (LinkedIn, Naukri, Instahyre, Cutshort, company career page). Playwright scrapes the full JD, requirements, and company info.
- **Autonomous Company Research:** An AI agent independently researches the company — recent news, funding status, Glassdoor sentiment, employee count trends, tech stack from engineering blogs.
- **AI Resume Tailoring:** For each application, the agent rewrites the user's resume to highlight the specific skills that job wants. Confidence-scored, user-reviewable.
- **Cover Letter Generation:** Personalized cover letters referencing actual company details (funding, mission, recent products) — not generic templates.
- **Application Dashboard:** Visual pipeline showing applied, in-review, interview, offer, rejected. Filter by company, role, location, salary range.
- **Proactive Company Monitoring:** After applying, the agent silently watches applied companies for layoffs, leadership changes, and hiring surges. Emails the user when something important happens.
- **AI Chat Advisor:** Users ask questions like 'should I apply to this company?' or 'which 3 of my applications are most likely to convert?' — the agent answers using their actual application data.
- **Weekly Agent Reports:** Every Sunday, the agent emails a summary: applications sent, responses received, companies to watch, recommended focus areas for the coming week.
- **Multi-user Auth:** GitHub, Google, and email magic link authentication.
- **Freemium with Razorpay:** Free tier with sensible limits; Pro tier via Razorpay subscription.

WHY THIS PROJECT FITS YOUR CAREER GOALS

You are the target user. As a junior developer aiming to jump from ₹27K to ₹55-80K, you will apply to 50-100 jobs in the next year. You will *use* this product, find real bugs, and have genuine interview stories.

It proves agentic AI skills. The research agent makes decisions, uses tools, handles conflicting data, and runs autonomously. This is the 2026 hiring keyword.

It covers the full stack. Frontend, backend, database, DevOps, payments, AI — one project, every skill.

It tells a story interviewers remember. 'I built a tool that caught silent layoffs at one of my target companies by monitoring their LinkedIn employee count' is unforgettable.

2 Why This Project Stands Out

Portfolio projects are judged on distinctiveness, technical depth, and execution quality. Here is the honest comparison against the category of AI projects juniors typically submit:

Aspect	Typical AI Project	This Project
Interaction	Single chat prompt	Multi-step agent with tool use
Data source	User text input	Live scraped data from multiple sources
State	Stateless	Stateful — tracks apps over time
Autonomy	User triggers every action	Agent runs on schedule without user
Monetization	None	Real Razorpay subscriptions
Infra	Runs on Vercel only	Docker + Nginx + VPS + CDN
Story	I wrapped an LLM API	I solved my own problem while job hunting

Skill coverage at a glance:

Skill Area	Coverage	Depth
Frontend (React, Next.js, TypeScript)	95%	Intermediate → Advanced
Backend (APIs, auth, validation)	95%	Intermediate → Advanced
Database (Postgres, Prisma, indexing)	90%	Intermediate
DevOps (Docker, Nginx, cloud)	85%	Intermediate (rare for juniors)
AI Integration (LLM APIs, streaming)	95%	Advanced for a junior
Agentic AI (planning, tool use, scheduling)	85%	Intermediate — genuinely agentic
Payments (webhooks, subscriptions)	75%	Intermediate
System Design (real tradeoffs)	70%	Basic → Intermediate
Applied DSA	60%	Practical, not competitive

3 Complete Tech Stack Breakdown

Every technology below is either free forever or has a generous free tier sufficient for this project. No credit card is required anywhere except Oracle Cloud (for verification only) and Razorpay (for receiving real payouts, not for building).

Frontend

Technology	Why This	What You Learn
Next.js 14 (App Router)	Industry standard, SSR + API in one	Server components, routing, middleware
TypeScript (strict)	Catches bugs, expected in interviews	Types, generics, type guards
Tailwind CSS	Rapid styling, no CSS files	Utility-first CSS, responsive design
shadcn/ui	Professional components, copy-paste	Component composition, accessibility
Recharts	Best free React charting	Data visualization, SVG rendering
React Hook Form + Zod	Form handling with validation	Schema validation, controlled inputs
Vercel AI SDK	Streaming AI UX	Server-Sent Events, streaming

Backend & Database

Technology	Why This	Free Tier
Next.js API Routes	No separate backend	Included
Prisma ORM	Type-safe DB, auto migrations	Open source
PostgreSQL (NeonDB)	Relational DB, interview essential	0.5 GB forever
Redis (Upstash)	Cache, queue, rate limit	10K commands/day
BullMQ	Background job processing	Open source
NextAuth.js (Auth.js v5)	OAuth + magic links	Open source
Resend	Transactional emails	3,000/month
Razorpay	Payment processing (India)	Free to integrate

AI & Agentic Layer

Technology	Purpose	Free Tier
Groq API (Llama 3.3 70B)	Primary LLM — research, tailoring, chat	30 req/min, 14.4K/day
Gemini Flash (Google)	Backup LLM, vision for OCR	Generous free tier
Vercel AI SDK	Streaming responses in React	Open source
Playwright	Headless browser scraping	Open source
Cheerio	Lightweight HTML parsing	Open source
pdf-parse	Resume PDF extraction	Open source
Structured JSON mode	Force typed AI output	Part of Groq API

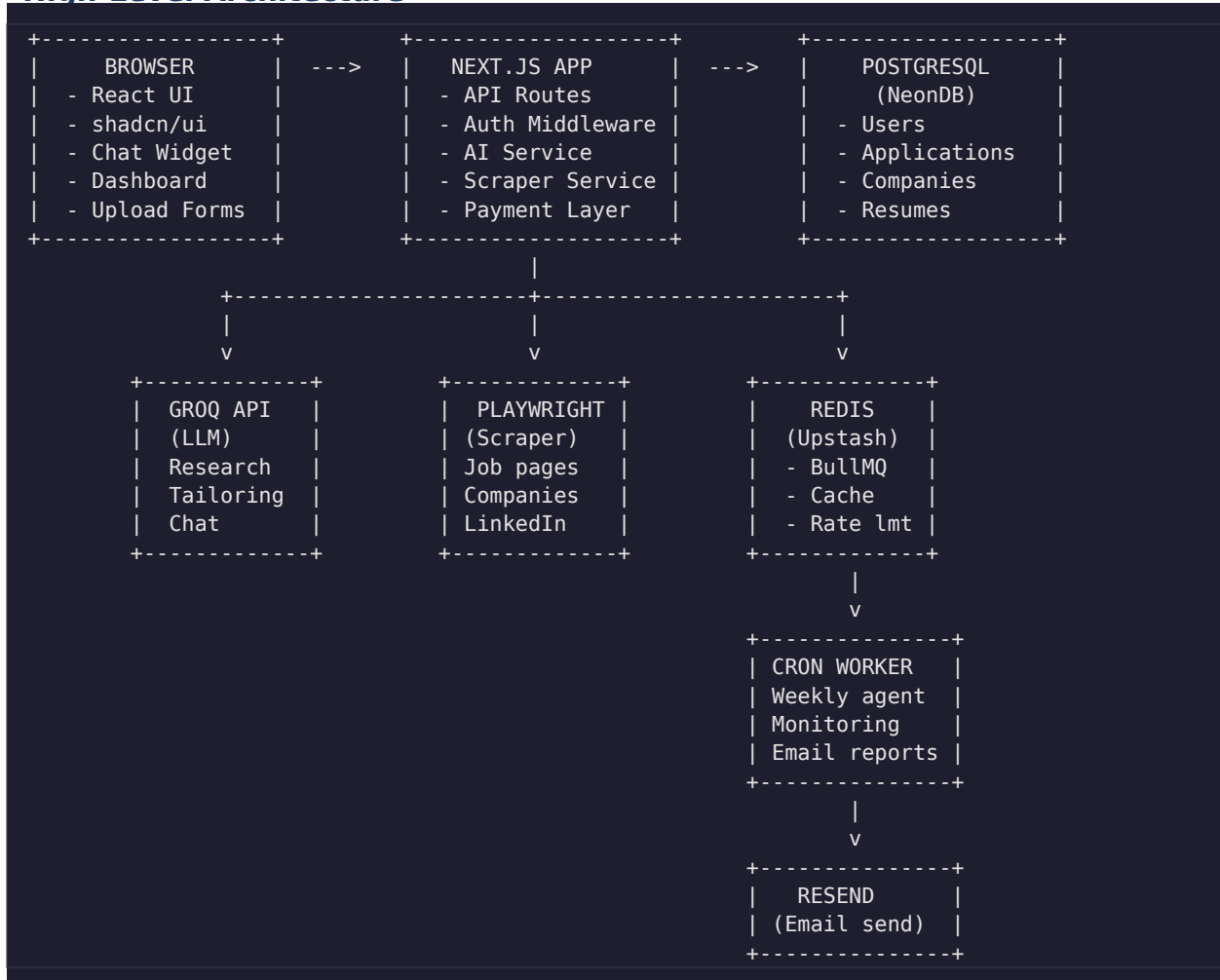
DevOps & Deployment

Technology	Purpose	Free Tier
Docker + Compose	Containerize app, worker, DB, Redis	Free, open source
Nginx	Reverse proxy, SSL, caching	Free, open source
Oracle Cloud VM	Production server (ARM, 6 GB RAM)	Always free
Cloudflare	DNS, CDN, SSL, DDoS	Free plan
GitHub Actions	CI/CD pipeline	Free for public repos
Let's Encrypt	SSL certificates	Free forever
Sentry	Production error tracking	5K errors/month
PostHog	Product analytics	Free tier

4 System Architecture & Data Flow

Understanding the architecture before writing code is what separates juniors from mid-level engineers. Every arrow below represents real code you will write.

High-Level Architecture



Request Flow — How Data Moves

- **Resume Upload Flow:** User uploads resume → parsed by pdf-parse → sent to Groq for structured extraction → stored as structured JSON in Postgres → indexed for retrieval.
- **Application Flow:** User pastes job URL → queued in BullMQ → Playwright scrapes page → Groq extracts JD details → triggers company research sub-job → merges results → stores Application record → notifies frontend.
- **Research Agent Flow:** Research job picks up company name → checks Redis cache → if miss, scrapes LinkedIn, Glassdoor, company blog, news → each source returns partial data → Groq synthesizes into final report with confidence scores → caches for 24 hours.
- **Chat Flow:** User asks question → server loads user's applications + research context → constructs prompt → streams Groq response via SSE → frontend renders word-by-word → saves to ChatMessage table.

- **Monitoring Agent Flow:** BullMQ cron fires every Sunday 9 AM → worker loads pro users → for each user: re-scrapes their applied companies → compares against baseline → if significant change (layoff, funding, mass exit), generates alert → sends via Resend → logs to WeeklyReport.
- **Payment Flow:** User clicks Upgrade → Razorpay checkout → webhook hits /api/razorpay/webhook → verifies signature → updates Subscription record → flips user's pro flag → feature limits lift immediately.

5 Database Schema Design

This is your Prisma schema. Study each model — in interviews, explaining your database design decisions shows real engineering thinking.

```
model User {
  id          String    @id @default(cuid())
  name        String?
  email       String    @unique
  emailVerified DateTime?
  image       String?
  plan        Plan      @default(FREE)
  accounts    Account[]
  sessions    Session[]
  resumes     Resume[]
  applications Application[]
  subscription Subscription?
  chatMessages ChatMessage[]
  weeklyReports WeeklyReport[]
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

enum Plan { FREE PRO }

model Resume {
  id          String    @id @default(cuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id])
  title       String    // "Backend focused", "Frontend focused"
  rawText     String    @db.Text
  parsedData  Json      // skills, experience, education (structured)
  fileUrl     String?
  isDefault   Boolean   @default(false)
  applications Application[]
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  @@index([userId])
}

model Company {
  id          String    @id @default(cuid())
  name        String
  domain      String?   @unique
  linkedinUrl String?
  researchData Json?    // funding, news, sentiment, risk flags
  researchScore Float?  // 0-10, higher = safer to apply
  lastResearchAt DateTime?
  applications Application[]
  @@index([name])
}
```

Database Schema (continued)

```
model Application {
  id          String      @id @default(cuid())
  userId      String
  user        User        @relation(fields: [userId], references: [id])
  companyId   String
  company     Company     @relation(fields: [companyId], references: [id])
  resumeId    String?
  resume      Resume?     @relation(fields: [resumeId], references: [id])
  jobUrl      String
  jobTitle    String
  jobLocation String?
  salaryRange String?
  jobDescription String    @db.Text
  extractedSkills Json      // AI-extracted required skills
  matchScore  Float        // how well user matches the job
  tailoredResume String?   @db.Text
  coverLetter String?      @db.Text
  status      AppStatus    @default(APPLIED)
  appliedAt   DateTime?
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
  @@index([userId, status])
  @@index([userId, createdAt])
  @@index([companyId])
}

enum AppStatus {
  DRAFT APPLIED IN_REVIEW INTERVIEW OFFER REJECTED WITHDRAWN
}
```

```

model Subscription {
  id          String    @id @default(cuid())
  userId      String    @unique
  user        User      @relation(fields: [userId], references: [id])
  razorpaySubId String?  @unique
  razorpayCustomerId String?
  status      String    // active, cancelled, paused, halted
  currentPeriodEnd DateTime?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model ChatMessage {
  id          String    @id @default(cuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id])
  role        String    // "user" or "assistant"
  content     String    @db.Text
  metadata    Json?
  createdAt   DateTime @default(now())
  @@index([userId, createdAt])
}

model WeeklyReport {
  id          String    @id @default(cuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id])
  weekStart   DateTime
  weekEnd     DateTime
  applicationsSent Int
  responsesCount Int
  companiesWatched Int
  alerts      Json      // [{company, type, severity, message}]
  aiSummary   String    @db.Text
  emailSentAt DateTime?
  createdAt   DateTime @default(now())
}

```

WHY THESE DESIGN DECISIONS MATTER IN INTERVIEWS

- Composite indexes on (userId, status) and (userId, createdAt) show query performance awareness.
- Separating Company from Application lets multiple users share research data — saves API calls.
- researchScore as a float enables sorting 'safest companies first' in the dashboard.
- Subscription is its own model, not a boolean on User — supports complex billing states.
- matchScore shows the AI is not a black box; users see how well they fit a job.
- enum AppStatus enforces a clean state machine for application tracking.

6 Free vs Pro Feature Split

The freemium model is designed to give free users enough value to stick around while creating natural pressure to upgrade. The Pro tier unlocks exactly the features that are either expensive to run (AI monitoring) or deliver outsized value (weekly reports, company alerts).

Feature	Free	Pro (₹199/mo)
Applications tracked	10	Unlimited
AI company research	3 / month	Unlimited
Resume tailoring	5 / month	Unlimited
Cover letter generation	5 / month	Unlimited
Multiple resume profiles	1	Unlimited
Company monitoring after applying	No	Yes
Weekly AI agent emails	No	Yes
Layoff / news alerts	No	Yes
Interview prep generator	No	Yes
Application analytics	Basic	Full dashboard
Data export (CSV + PDF)	No	Yes
AI chat messages	10 / month	Unlimited
Priority LLM model	Standard	Llama 3.3 70B
Support	Community	Priority email

Pricing Logic

Plan	Monthly	Yearly (save ~17%)
Free	₹0	₹0
Pro	₹199	₹1,999

One-line pitch per tier:

Free: Track your job search.

Pro: Your AI job search assistant that works while you sleep.

PHASE 1 — Project Setup & Foundation

Week 1-2 | Setting up the entire development environment

This phase establishes your foundation. Do not rush — a clean setup prevents hours of debugging later. By the end of Week 2, you should be able to log in, see an empty dashboard, and have Postgres + Redis running locally in Docker.

Day 1-2: Initialize & Install

- **Create Next.js project:** `npx create-next-app@latest job-agent --typescript --tailwind --app --src-dir`
- **Install core deps:** `npm install prisma @prisma/client next-auth @auth/prisma-adapter`
- **Initialize Prisma:** `npx prisma init` and paste the schema from Section 5.
- **Install UI layer:** `npx shadcn@latest init` then add button, card, input, dialog, dropdown-menu, tabs, table.
- **Install AI stack:** `npm install ai @ai-sdk/groq groq-sdk zod`

Prisma Client Singleton (src/lib/prisma.ts):

```
import { PrismaClient } from '@prisma/client'

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined
}

export const prisma = globalForPrisma.prisma ?? new PrismaClient()

if (process.env.NODE_ENV !== 'production') {
  globalForPrisma.prisma = prisma
}
```

Day 3-5: Authentication

- Configure NextAuth with GitHub + Google providers and Prisma adapter.
- Add email magic link support via Resend (optional but nice UX).
- Create `src/middleware.ts` to protect `/dashboard/*` routes.
- Build login page with three login options and loading states.
- Build dashboard shell: sidebar, header with avatar + logout, main area.

Day 6-10: Docker Compose & Local DB

- Install Docker Desktop. Verify with `docker --version`.
- Create `docker-compose.yml` with postgres and redis services.
- Run `docker compose up -d`, verify with `docker compose ps`.
- Point `DATABASE_URL` to local Docker Postgres for development, NeonDB for production.
- Run `npx prisma db push` to apply schema.

```
# docker-compose.yml
version: '3.8'
services:
  postgres:
    image: postgres:16-alpine
    container_name: jobagent-postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: localpass
      POSTGRES_DB: jobagent
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    container_name: jobagent-redis
    ports:
      - "6379:6379"

volumes:
  postgres_data:
```

CHECKPOINT — END OF PHASE 1

You should have: running Next.js app, working login with GitHub + Google, Postgres + Redis in Docker, Prisma schema applied, protected dashboard layout. Fix everything before moving to Phase 2.

PHASE 2 — Resume Upload & Parsing

Week 3-4 | PDF/DOCX parsing, structured extraction with AI

This phase turns messy resume files into clean structured data that the AI agent will use throughout the rest of the project. Your earlier PDF experience helps directly here.

Week 3: Resume Upload UI

- Build drag-and-drop upload with react-dropzone.
- Support PDF and DOCX. Limit file size to 5 MB.
- Store file in Cloudinary (free tier) or UploadThing — 2 GB free.
- Show upload progress and parsing status.
- Allow multiple resumes per user (backend / frontend / data-focused variants).

Week 4: Structured Parsing with Groq

- Extract raw text: pdf-parse for PDF, mammoth for DOCX.
- Send raw text to Groq with a strict Zod schema for structured output.
- Extract: name, email, phone, summary, skills[], experience[], education[], projects[].
- Validate AI response with Zod. Retry with adjusted prompt on invalid JSON.
- Store parsed JSON in Resume.parsedData. Display parsed view for user confirmation.
- Let user edit the parsed data — this is the source of truth for tailoring.

```
import { z } from 'zod'

export const parsedResumeSchema = z.object({
  name: z.string(),
  email: z.string().email().optional(),
  phone: z.string().optional(),
  summary: z.string().optional(),
  skills: z.array(z.string()),
  experience: z.array(z.object({
    company: z.string(),
    role: z.string(),
    startDate: z.string(),
    endDate: z.string().nullable(),
    description: z.string(),
    technologies: z.array(z.string()).optional(),
  })),
  education: z.array(z.object({
    institution: z.string(),
    degree: z.string(),
    year: z.string(),
  })),
  projects: z.array(z.object({
    name: z.string(),
    description: z.string(),
    technologies: z.array(z.string()),
    link: z.string().url().optional(),
  })),
})

export type ParsedResume = z.infer<typeof parsedResumeSchema>
```

CHECKPOINT — END OF PHASE 2

Upload a PDF resume, see it parsed into structured fields, edit any wrong values, save to database, and support multiple resume profiles per user.

PHASE 3 — Job Scraping & Research Agent

Week 5-7 | The autonomous core of your project

This is the most impressive phase. You build an agent that scrapes job pages, independently researches companies, and returns a confidence-scored risk assessment. This is where 'agentic AI' stops being a buzzword in your project.

Week 5: Playwright Scraper Service

- Install Playwright: `npm install playwright` then `npx playwright install chromium`.
- Build job URL parser that detects source (LinkedIn, Naukri, Instahyre, custom career page).
- Per-source adapters using the Strategy Pattern — one function per job board.
- Extract: job title, company, location, salary (if visible), JD text, required skills, experience level.
- Handle anti-bot basics: realistic user agent, viewport size, small delays between requests.
- Respect robots.txt and rate limit yourself to 1 request / 3 seconds per domain.

Week 6: Multi-Source Company Research

- Given a company name, research in parallel: company website about page, LinkedIn company page, Glassdoor (via search), recent news headlines (via news search API or scraping Google News).
- Each source returns a partial `CompanyInsight` object. Cache in Redis for 24 hours.
- Send all partials to Groq with a synthesis prompt: 'Combine these into a final report. Flag risks.'
- Output: funding status, employee count trend, sentiment score, recent news summary, risk flags (layoffs, leadership exits, mass hiring halt).
- Store in `Company.researchData` and `researchScore` (0-10, higher = safer).

```
// src/lib/agents/company-research.ts
export async function researchCompany(name: string): Promise<CompanyInsight> {
  const cached = await redis.get(`company:${name}`)
  if (cached) return JSON.parse(cached)

  // Fan out to multiple sources in parallel
  const [website, linkedin, news, glassdoor] = await Promise.allSettled([
    scrapeCompanyWebsite(name),
    scrapeLinkedIn(name),
    searchRecentNews(name),
    scrapeGlassdoor(name),
  ])

  // Collect whatever succeeded
  const partials = {
    website: website.status === 'fulfilled' ? website.value : null,
    linkedin: linkedin.status === 'fulfilled' ? linkedin.value : null,
    news: news.status === 'fulfilled' ? news.value : null,
    glassdoor: glassdoor.status === 'fulfilled' ? glassdoor.value : null,
  }

  // Let the LLM synthesize – it handles conflicting info
  const synthesis = await groq.chat.completions.create({
    model: 'llama-3.3-70b-versatile',
    response_format: { type: 'json_object' },
    temperature: 0.2,
    messages: [
      { role: 'system', content: RESEARCH_SYSTEM_PROMPT },
      { role: 'user', content: `Data: ${JSON.stringify(partials)}` },
    ],
  })

  const insight = companyInsightSchema.parse(
    JSON.parse(synthesis.choices[0].message.content!)
  )

  await redis.set(`company:${name}`, JSON.stringify(insight), 'EX', 86400)
  return insight
}
```

Week 7: Match Scoring & Application Creation

- Given a scraped JD and a user's parsed resume, compute a match score (0-100).
- Extract required skills from JD using Groq. Compare against user's skills. Weight by experience.
- Create Application record with match score, extracted skills, company research data.
- Display results: 'You match 72% for this role. Your strong areas: X, Y. Gaps: Z.'

WHAT MAKES THIS AGENTIC

1. **Planning:** The agent decides which sources to check based on what's available.
2. **Tool use:** Playwright, HTTP, Redis, Groq — the agent uses multiple tools.
3. **Handling conflicts:** When sources disagree, Groq synthesizes a final answer with a confidence score.
4. **Caching & efficiency:** The agent avoids re-research within 24 hours — a real engineering concern.
5. **Failure recovery:** If 2 of 4 sources fail, the agent still produces a reasonable answer.

PHASE 4 — Dashboard & Analytics

Week 8-9 | React hooks, Recharts, pipeline visualization

This phase levels up your React skills. Building a real dashboard forces you to understand re-renders, memoization, and data transformation at scale.

Components to Build

- **Summary cards:** Total applications, response rate, interview rate, top match category. Use `useMemo`.
- **Pipeline Kanban:** Columns for each `AppStatus`. Drag-and-drop to update status. Use `@dnd-kit`.
- **Match score distribution:** Recharts histogram showing how well you match the jobs you're applying to.
- **Company risk heatmap:** Color-coded grid showing which applied companies have risk flags.
- **Application timeline:** Line chart of applications sent per week vs responses received.
- **Top skills in demand:** Extracted from all your applied JDs — tells you what to learn next.
- **Table with filters:** Sort by date, match score, status; filter by company, role, location.

```
'use client'
import { useState, useMemo, useCallback } from 'react'

export function Dashboard({ applications }: Props) {
  const [statusFilter, setStatusFilter] = useState<AppStatus | 'ALL'>('ALL')
  const [search, setSearch] = useState('')

  // useMemo: avoid recomputing on every keystroke
  const filtered = useMemo(() => {
    return applications.filter(a => {
      if (statusFilter !== 'ALL' && a.status !== statusFilter) return false
      if (search && !a.company.name.toLowerCase().includes(search.toLowerCase()))
        return false
      return true
    })
  }, [applications, statusFilter, search])

  // useCallback: stable reference so child chart doesn't re-render
  const handleStatusChange = useCallback((status: AppStatus | 'ALL') => {
    setStatusFilter(status)
  }, [])

  const stats = useMemo(() => ({
    total: filtered.length,
    responded: filtered.filter(a => a.status !== 'APPLIED').length,
    interviews: filtered.filter(a => a.status === 'INTERVIEW').length,
    avgMatch: filtered.reduce((s, a) => s + a.matchScore, 0) / (filtered.length || 1),
  }), [filtered])

  return (/* ... */)
}
```

WHY HOOKS MATTER HERE (AND IN INTERVIEWS)

When asked 'explain useCallback' say: *'In my job agent dashboard, the Kanban board re-rendered on every keystroke in the search box because the drag handler was recreated every render. useCallback gave it a stable reference and fixed the perf issue. Same for useMemo on the filter logic — filtering 200 applications on every keystroke was visibly janky until I memoized it.'*

PHASE 5 — AI Chat & Resume Tailoring

Week 10-12 | Conversational AI with real application context

The most impressive user-facing feature — a chat assistant that knows the user's actual job search data and gives contextual advice, plus an AI tailor that rewrites resumes per job.

Week 10-11: Resume Tailoring Engine

- Given user's parsed resume + a specific job's extracted skills, prompt Groq to rewrite the summary and reorder experience bullets to highlight matching skills.
- Never fabricate — only reorder, rephrase, and emphasize what's already there. This is critical for integrity.
- Show a diff view: original vs tailored. Let user accept / reject individual changes.
- Generate cover letter referencing specific company details (funding, mission, recent product).
- Store tailored resume + cover letter on the Application record.

Week 12: Streaming Chat with Context Injection

- Use Vercel AI SDK's useChat hook — gives streaming, message history, loading states for free.
- Before sending user's message to Groq, load their full application context from DB.
- Construct system prompt with: total apps, status breakdown, top 5 recent applications, company risk flags.
- System prompt: *'You are a job search advisor with access to this user's real application data. Use specific numbers. Be direct.'*
- Save every message to ChatMessage table for history and rate limiting.
- Enforce Free tier limit: 10 messages / month. Show upgrade prompt on limit.

```
// src/app/api/chat/route.ts
import { streamText } from 'ai'
import { createGroq } from '@ai-sdk/groq'
import { prisma } from '@lib/prisma'
import { getServerSession } from 'next-auth'

const groq = createGroq({ apiKey: process.env.GROQ_API_KEY! })

export async function POST(req: Request) {
  const session = await getServerSession()
  if (!session?.user?.id) return new Response('Unauthorized', { status: 401 })

  const { messages } = await req.json()

  const [apps, user] = await Promise.all([
    prisma.application.findMany({
      where: { userId: session.user.id },
      include: { company: true },
      orderBy: { createdAt: 'desc' },
      take: 20,
    }),
    prisma.user.findUnique({ where: { id: session.user.id } }),
  ])

  const context = `
  USER JOB SEARCH CONTEXT:
  Total applications: ${apps.length}
  By status: ${summarizeByStatus(apps)}
  Recent applications:
  ${apps.slice(0, 5).map(a =>
    `- ${a.jobTitle} at ${a.company.name} | match ${a.matchScore}% | ${a.status}`
  )}.join('\n')}
  Companies with risk flags: ${apps.filter(a => a.company.researchScore! < 5).length}
  `

  const result = streamText({
    model: groq('llama-3.3-70b-versatile'),
    system: `You are a job search advisor. Real data below. Use specific numbers.\n${context}`,
    messages,
  })

  return result.toDataStreamResponse()
}
```

EXAMPLE CHAT INTERACTIONS (DEMO GOLD)

User: Which of my applications should I follow up on this week?

Agent: Based on your 20 applications, I'd prioritize these 3: Acme Corp (applied 10 days ago, match 82%, no response), Beta Ltd (interview scheduled, confirm attendance), Gamma Inc (new funding round 3 days ago, good time to reach out).

User: Am I applying to too many risky companies?

Agent: Yes — 7 of your 20 applications (35%) are at companies with risk scores below 5. Two had recent layoffs. Consider shifting focus to safer companies.

PHASE 6 — Weekly Monitoring Agent

Week 13-14 | Fully autonomous BullMQ + Resend email pipeline

Until now, your agent responds when asked. Now you build an agent that wakes up on its own schedule, re-researches applied companies, detects changes, and sends the user an email. **This is the agentic AI story that wins interviews.**

Week 13: BullMQ Queue & Worker

- Install: `npm install bullmq ioredis`.
- Create queue at `src/lib/queue.ts`. Define job types and default retry config.
- Create worker at `src/workers/weekly-monitor.ts` — runs as a separate process.
- Schedule cron with BullMQ's `upsertJobScheduler` — every Sunday 9 AM.
- **DSA connection:** BullMQ is literally a FIFO queue backed by Redis. When you study queues in DSA, you already understand them.

```
// src/lib/queue.ts
import { Queue } from 'bullmq'
import IORedis from 'ioredis'

const connection = new IORedis(process.env.REDIS_URL!, {
  maxRetriesPerRequest: null,
})

export const weeklyQueue = new Queue('weekly-monitor', {
  connection,
  defaultJobOptions: {
    removeOnComplete: 100,
    removeOnFail: 50,
    attempts: 3,
    backoff: { type: 'exponential', delay: 60000 },
  },
})

export async function scheduleWeeklyMonitor() {
  await weeklyQueue.upsertJobScheduler(
    'weekly-company-monitor',
    { pattern: '0 9 * * 0' }, // Every Sunday 9 AM
    { name: 'monitor-all-pro-users' }
  )
}
```

Week 14: Monitor Logic & Email Delivery

- Worker loads all Pro users with recent applications (last 60 days).
- For each user: loads their applied companies → re-runs company research → compares new data vs stored `researchData`.
- Detects change types: `employee_count_drop_over_5_percent`, `funding_announced`, `leadership_exit`, `layoff_news`, `mass_hiring_halt`.
- For each significant change, emits an Alert object with severity (info, warn, critical).
- Groq synthesizes alerts into a personalized email with 1-paragraph summary + top 3 actions.
- Resend sends the HTML email. `WeeklyReport` record saved for history.

THIS IS AGENTIC AI — UNDERSTAND WHAT YOU BUILT

Your weekly monitor is an autonomous agent that follows the classic loop:

- 1. PERCEIVES** — loads applied companies and re-scrapes their current state.
- 2. THINKS** — compares new vs old, identifies meaningful changes, asks the LLM to summarize.
- 3. ACTS** — sends alerts via email, saves report, updates `Company.researchData`.
- 4. REPEATS** — runs automatically every week, without human trigger.

In interviews, use this concrete example to explain the perceive-think-act loop. It's the simplest form of an AI agent, but the pattern is identical to what's used in complex systems.

PHASE 7 — Razorpay Payments & Subscriptions

Week 15-16 | Real production payments with webhooks

Payments separate toy projects from real products. This phase teaches webhooks, idempotency, and subscription state machines — senior-level topics most juniors never touch.

Week 15: Razorpay Setup & Checkout

- Sign up at razorpay.com. Activate Test Mode (no bank details needed to start).
- Get test API keys (KEY_ID + KEY_SECRET). Add to .env.
- Install SDK: `npm install razorpay`.
- Create a Plan in Razorpay dashboard: Pro Monthly ₹199, Pro Yearly ₹1,999.
- Build pricing page with Free / Pro cards. Upgrade button creates a Razorpay Subscription.
- Frontend loads Razorpay Checkout.js, opens modal, handles success/failure.

Week 16: Webhook Handler & Subscription State

- Create `/api/razorpay/webhook` endpoint. Verify signature using webhook secret.
- Handle events: `subscription.activated`, `subscription.charged`, `subscription.halted`, `subscription.cancelled`.
- **Idempotency:** Use the event ID as a dedupe key — Razorpay retries webhooks on failure.
- Update `User.plan` and `Subscription.status` based on events.
- Build middleware that checks `user.plan === 'PRO'` before allowing Pro features.
- Build billing page: current plan, next charge date, cancel subscription button.
- Test end-to-end in Test Mode with Razorpay's test cards.

```
// src/app/api/razorpay/webhook/route.ts
import crypto from 'crypto'
import { prisma } from '@lib/prisma'

export async function POST(req: Request) {
  const body = await req.text()
  const signature = req.headers.get('x-razorpay-signature')!

  const expected = crypto
    .createHmac('sha256', process.env.RAZORPAY_WEBHOOK_SECRET!)
    .update(body)
    .digest('hex')

  if (signature !== expected) {
    return new Response('Invalid signature', { status: 401 })
  }

  const event = JSON.parse(body)
  const eventId = event.id

  // Idempotency: skip if we've already processed this event
  const existing = await prisma.webhookEvent.findUnique({ where: { eventId } })
  if (existing) return new Response('OK', { status: 200 })

  await prisma.webhookEvent.create({ data: { eventId, type: event.event } })

  switch (event.event) {
    case 'subscription.activated':
    case 'subscription.charged':
      await activateUser(event.payload.subscription.entity)
      break
    case 'subscription.halted':
    case 'subscription.cancelled':
      await downgradeUser(event.payload.subscription.entity)
      break
  }

  return new Response('OK', { status: 200 })
}
```

INTERVIEW GOLD

Webhooks + idempotency is a senior-level topic. When interviewers hear *'I handled Razorpay webhooks with HMAC signature verification and idempotent processing using event IDs as dedupe keys'*, they immediately know you've worked with real production systems. This one phase alone elevates your candidacy meaningfully.

PHASE 8 — Docker, Nginx & Deployment

Week 17-18 | Production deployment on Oracle Cloud with Cloudflare

This phase transforms you from 'can build React apps' to 'can deploy production systems'. Most juniors never learn this — massive interview advantage.

Day 1-2: Multi-Stage Dockerfiles

- Multi-stage build for the Next.js app. Final image around 200-300 MB.
- Separate Dockerfile for the BullMQ worker — smaller image, just Node + worker script.
- Use Next.js standalone output mode: set output: 'standalone' in next.config.js.

```
# Stage 1: Dependencies
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

# Stage 2: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npx prisma generate && npm run build

# Stage 3: Runtime
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
RUN addgroup --system --gid 1001 nodejs \
  && adduser --system --uid 1001 nextjs
COPY --from=builder /app/public ./public
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./next/static
USER nextjs
EXPOSE 3000
CMD ["node", "server.js"]
```

Day 3: Nginx Configuration

```
upstream nextjs_app { server app:3000; }

server {
    listen 80;
    server_name yourdomain.com;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name yourdomain.com;

    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;

    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Strict-Transport-Security "max-age=63072000" always;

    gzip on;
    gzip_types text/plain text/css application/json application/javascript;

    limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;

    location /api/ {
        limit_req zone=api burst=20 nodelay;
        proxy_pass http://nextjs_app;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /_next/static/ {
        proxy_pass http://nextjs_app;
        expires 365d;
        add_header Cache-Control "public, immutable";
    }

    location / {
        proxy_pass http://nextjs_app;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Day 4-5: Oracle Cloud + Cloudflare + SSL

- Sign up for Oracle Cloud Always Free tier. Create an ARM Ampere A1 VM (1 OCPU, 6 GB RAM, Ubuntu 22.04).
- **Fallback if Oracle capacity unavailable:** Railway, Render, or Hetzner EUR 4/mo.
- SSH in, install Docker + docker-compose, clone your repo, configure .env.production, run `docker compose -f docker-compose.prod.yml up -d`.
- Cloudflare: add domain → point A record to VM IP → enable Proxied mode → set SSL to Full (Strict).
- Install certbot, run `certbot certonly --standalone -d yourdomain.com`, mount certs into Nginx container.
- Oracle firewall: ingress rules for ports 80, 443. VM iptables: `sudo iptables -I INPUT -p tcp --dport 80 -j ACCEPT`.

- Test end-to-end: login, upload resume, submit job URL, check agent runs, trigger a test payment.

Day 6-7: CI/CD with GitHub Actions

- Create `.github/workflows/deploy.yml`.
- On push to main: run tests, build Docker images, push to GitHub Container Registry.
- SSH into VM, pull latest images, restart containers. Zero-downtime with rolling restart.
- Add basic smoke test after deploy — check / returns 200.

CHECKPOINT — PROJECT SHIPPED

You now have a production SaaS at <https://yourdomain.com> with: SSL, CDN, DDoS protection, rate limiting, error monitoring, analytics, automated deployments, and real payments. This is the skill set of a mid-level engineer, not a junior. Celebrate.

15 API Endpoints Reference

Method	Endpoint	Purpose	Auth
POST	/api/auth/[...nextauth]	Authentication (NextAuth)	No
POST	/api/resumes/upload	Upload resume, parse with AI	Yes
GET	/api/resumes	List user's resumes	Yes
PATCH	/api/resumes/:id	Edit parsed resume data	Yes
POST	/api/applications	Submit job URL, scrape, research, create	Yes
GET	/api/applications	List with filters (status, company)	Yes
PATCH	/api/applications/:id	Update status	Yes
POST	/api/applications/:id/tailor	AI-tailor resume for this job	Yes
POST	/api/applications/:id/cover-letter	Generate cover letter	Yes
GET	/api/companies/:id	Company research report	Yes
POST	/api/companies/:id/refresh	Force re-research	Pro
POST	/api/chat	Streaming AI advisor	Yes
GET	/api/chat/history	Past conversations	Yes
GET	/api/reports	Weekly report history	Pro
POST	/api/reports/trigger	Manually trigger report (dev)	Yes
POST	/api/razorpay/checkout	Create subscription order	Yes
POST	/api/razorpay/webhook	Handle subscription events	Signature
GET	/api/billing	Current plan + next charge	Yes

16 Folder Structure

```
job-agent/
├── .github/workflows/deploy.yml      # CI/CD pipeline
├── docker/
│   ├── nginx/nginx.conf             # Nginx configuration
│   ├── Dockerfile                   # App Dockerfile (multi-stage)
│   └── Dockerfile.worker             # Worker Dockerfile
├── prisma/
│   ├── schema.prisma                # Database schema
│   └── seed.ts                      # Seed data
├── src/
│   ├── app/
│   │   ├── api/
│   │   │   ├── auth/[...nextauth]/route.ts
│   │   │   ├── resumes/{route,upload}/route.ts
│   │   │   ├── applications/{route,[id]/route,[id]/tailor/route}.ts
│   │   │   ├── companies/{[id]/route,[id]/refresh/route}.ts
│   │   │   ├── chat/route.ts
│   │   │   ├── reports/route.ts
│   │   │   └── razorpay/{checkout/route,webhook/route}.ts
│   │   ├── dashboard/
│   │   │   ├── layout.tsx           # Dashboard shell
│   │   │   ├── page.tsx             # Overview + pipeline
│   │   │   ├── applications/page.tsx
│   │   │   ├── resumes/page.tsx
│   │   │   ├── companies/page.tsx
│   │   │   ├── chat/page.tsx
│   │   │   ├── reports/page.tsx
│   │   │   └── billing/page.tsx
│   │   ├── pricing/page.tsx
│   │   ├── login/page.tsx
│   │   └── page.tsx                 # Landing page
│   ├── components/
│   │   ├── ui/                     # shadcn components
│   │   ├── dashboard/{SummaryCards,Sidebar,Header}.tsx
│   │   ├── applications/{PipelineKanban,AppTable,AddAppDialog}.tsx
│   │   ├── companies/{RiskBadge,ResearchCard}.tsx
│   │   ├── resume/{ResumeUploader,ParsedResumeEditor}.tsx
│   │   └── chat/{ChatWindow,MessageBubble}.tsx
│   ├── lib/
│   │   ├── agents/{company-research,weekly-monitor,tailor}.ts
│   │   ├── scrapers/{linkedin,naukri,instahyre,generic}.ts
│   │   ├── ai/{groq-client,prompts,schemas}.ts
│   │   ├── razorpay/{client,webhook}.ts
│   │   └── prisma.ts | redis.ts | queue.ts | validations.ts
│   ├── workers/weekly-monitor.ts    # Separate process
│   ├── hooks/{useApplications,useChat,useBilling}.ts
│   ├── types/index.ts
│   └── middleware.ts
├── docker-compose.yml               # Local dev
├── docker-compose.prod.yml          # Production
├── .env.example
└── README.md
```

17 Environment Variables & Free Services Setup

```
# Database (NeonDB – free forever, 0.5 GB)
DATABASE_URL="postgres://user:pass@ep-xxx.neon.tech/jobagent"

# Redis (Upstash – free tier, 10K cmd/day)
REDIS_URL="rediss://default:xxx@xxx.upstash.io:6379"

# NextAuth
NEXTAUTH_SECRET="generate-with-openssl-rand-base64-32"
NEXTAUTH_URL="http://localhost:3000"

# GitHub OAuth
GITHUB_ID="your-github-oauth-app-id"
GITHUB_SECRET="your-github-oauth-app-secret"

# Google OAuth
GOOGLE_ID="your-google-client-id"
GOOGLE_SECRET="your-google-client-secret"

# Groq AI
GROQ_API_KEY="gsk_XXXXXXXXXXXX"

# Gemini (backup / vision)
GEMINI_API_KEY="XXXXXXXXXXXX"

# Resend Email
RESEND_API_KEY="re_XXXXXXXXXXXX"
RESEND_FROM="Job Agent <noreply@yourdomain.com>"

# Razorpay (Test Mode during build)
RAZORPAY_KEY_ID="rzp_test_XXXXXXXXXXXX"
RAZORPAY_KEY_SECRET="XXXXXXXXXXXX"
RAZORPAY_WEBHOOK_SECRET="XXXXXXXXXXXX"
RAZORPAY_PRO_MONTHLY_PLAN_ID="plan_XXXXXXXXXXXX"

# File storage (Cloudinary or UploadThing)
CLOUDINARY_CLOUD_NAME="xxx"
CLOUDINARY_API_KEY="xxx"
CLOUDINARY_API_SECRET="xxx"

# Monitoring
SENTRY_DSN="https://xxx@xxx.ingest.sentry.io/xxx"
POSTHOG_KEY="phc_xxx"

APP_URL="https://yourdomain.com"
```

How to Get Each Free Service

- **NeonDB:** neon.tech → sign up with GitHub → create project → copy connection string. 0.5 GB free forever, no card.
- **Upstash:** upstash.com → create Redis DB → free tier 10K cmd/day, 256 MB.
- **Groq:** console.groq.com → create API key. Llama 3.3 70B access. 30 req/min, 14,400/day.
- **Gemini:** ai.google.dev → create API key. Generous free tier with vision model.
- **Resend:** resend.com → verify domain → API key. 3,000 emails/month, 100/day.
- **GitHub OAuth:** github.com/settings/developers → New OAuth App.
- **Google OAuth:** console.cloud.google.com → OAuth 2.0 Client ID.

- **Razorpay:** razorpay.com → Test Mode from day one. Real KYC only needed to receive payouts.
- **Cloudinary:** cloudinary.com → 25 GB free storage, 25 GB bandwidth.
- **Oracle Cloud:** cloud.oracle.com → Always Free → Ampere A1 VM (6 GB RAM). Card for verify only.
- **Cloudflare:** cloudflare.com → add domain → free plan → update nameservers.
- **Sentry:** sentry.io → free tier 5K errors/month.
- **PostHog:** posthog.com → free tier 1M events/month.

18 DSA Concepts You Will Learn Naturally

Instead of grinding LeetCode without context, this project teaches data structures through real usage.

DSA Concept	Where You Use It	Interview Answer
Hash Map	Grouping applications by status / company	'O(n) grouping of 200 apps vs O(n ²) nested loops.'
Queue (FIFO)	BullMQ job processing	'Redis-backed FIFO with exponential backoff retry.'
Priority Queue	Scheduling urgent company re-checks first	'Priority queue reorders work by recency and risk score.'
Binary Search	Finding applications in date ranges	'O(log n) range query on sorted timestamps.'
Tree	Skill hierarchy (JS → React → Next.js)	'Recursive traversal to aggregate at any skill level.'
Graph	Company relationships (parent, funding round participants)	'Adjacency list for company connections.'
Sliding Window	Detecting hiring velocity changes	'30-day window detects employee count drops > 5%.'
Reduce / Fold	Dashboard aggregations	'All dashboard metrics use Array.reduce — it is a fold.'
String Matching	Fuzzy company name matching	'Normalize + Levenshtein distance for name dedup.'
Caching / Memoization	Redis company research cache, useMemo	'24-hour TTL on research, memoized filters in UI.'

19 Common Mistakes & How to Avoid Them

Not using TypeScript strictly

Set `"strict": true` in `tsconfig.json` from day 1. Never use any. If you're tempted, it means you don't understand the data shape yet — figure it out first.

Putting scraping logic in API routes

Scraping belongs in a worker, not a request handler. A 20-second Playwright call will time out your API. Always enqueue a BullMQ job and return immediately with a job ID.

Not handling AI failures

LLMs WILL return garbage sometimes. Always: try/catch the API call, validate with Zod, have a fallback, log failures, retry with exponential backoff.

Storing secrets in code

Never commit `.env`. Add `.env*` to `.gitignore` on day 1. Use `.env.example` with placeholders. In production, set env vars via Docker Compose or the host, never in the image.

Not using database indexes

Without composite indexes, querying applications by `(userId, status)` takes seconds at 1000+ rows. The `@@index` directives in your schema are critical.

Running the worker inside Next.js

BullMQ worker must be a separate process. If you run it inside Next.js, a heavy scraping job blocks web requests. In Docker, the worker is a separate container.

Skipping webhook signature verification

If you don't verify Razorpay's webhook signature, anyone can hit your endpoint and fake a payment. Always HMAC-verify with the webhook secret.

Not making webhooks idempotent

Razorpay retries webhooks on any non-200 response. If you don't dedupe on event ID, you'll charge users twice or upgrade them twice.

Ignoring rate limits on scrapers

Hit LinkedIn 50 times in a minute and you'll get IP-banned. Always throttle to 1 req/3sec per domain. Use a shared Redis-backed rate limiter across workers.

Testing with 5 resumes and 3 jobs

Real bugs only appear at scale. Generate a script to create 100 fake applications and test with that. Performance issues in `useMemo`, slow queries, N+1 — they all surface at 100+ rows.

Over-engineering from the start

Build the simplest working version first. Don't add caching, rate limiting, or perf optimizations until you have a working app. Premature optimization wastes time.

Not deploying until the end

Deploy Phase 1 to production. Seriously. 'It works on my machine' is the most expensive phrase in software. Deploy early, deploy often, catch infra bugs while they're small.

20 Interview Talking Points from This Project

After completing this project, here is exactly what to say for common fullstack interview questions:

Interview Question	Your Answer
Tell me about a challenging project	Built an AI job-search agent that autonomously researches companies and monitors them after I apply. Hardest parts: the Playwright scraper with per-source strategy pattern, orchestrating the weekly monitor agent with BullMQ, and handling Razorpay webhooks idempotently.
How does authentication work?	NextAuth v5 with GitHub + Google OAuth and email magic links. Sessions in Postgres via Prisma adapter. Middleware protects /dashboard. Supports JWT, CSRF, secure cookies.
Explain useCallback / useMemo	In my Kanban pipeline, the drag handler was recreated every parent render, causing every card to re-render on every keystroke. useCallback fixed it. useMemo memoizes the filtered applications — filtering 200 items on every keystroke was visibly janky.
How do you deploy?	Multi-stage Docker build. Docker Compose orchestrates app, worker, Postgres, Redis, Nginx. Oracle Cloud ARM VM. Nginx handles SSL, gzip, rate limiting, static caching. Cloudflare for DNS, CDN, DDoS. CI/CD via GitHub Actions.
How does Docker work?	Each service in its own container. Compose defines a network so containers talk by service name (app calls postgres:5432). Volumes persist data. Multi-stage builds: 1 GB → 300 MB image.
Explain your database design	Relational schema: Users, Resumes, Applications, Companies, Subscriptions, ChatMessages, WeeklyReports. Composite indexes on (userId, status) and (userId, createdAt). JSON columns for flexible AI-extracted data.
What AI have you worked with?	Groq Llama 3.3 70B for research, resume tailoring, and streaming chat. Structured JSON output with Zod validation. Prompt engineering with low temperature for deterministic outputs. Multi-source fusion — LLM synthesizes data from Playwright scrapes across 4 sources into one report.
Describe an agentic system you built	My weekly monitor runs every Sunday. It perceives (re-scrapes applied companies), thinks (LLM compares vs baseline, flags risks), acts (sends email alerts, saves reports), repeats (cron). It catches silent layoffs by watching LinkedIn employee count week over week.
How do you handle payments?	Razorpay Subscriptions API. Checkout creates subscription, user pays, webhook fires. HMAC signature verification. Idempotent processing using Razorpay's event ID as a dedupe key — prevents double-charging on retries. Subscription state machine reflects active / halted / cancelled.

Interview Question	Your Answer
Background processing?	BullMQ on Redis. Weekly monitor as cron job in a separate Docker container. Retry with exponential backoff (1, 2, 4 minutes). Failed jobs kept for 50 generations for debugging.
How do you avoid getting IP-banned by scrapers?	Realistic user agent, viewport size, small randomized delays. Per-domain rate limiter in Redis — 1 req/3sec. Exponential backoff on 429s. Cache research for 24 hours so we don't rescrape the same company repeatedly.
DSA: explain queues	BullMQ is a FIFO queue backed by Redis lists — RPush to enqueue, BLPOP to dequeue. I also used a priority queue pattern to schedule urgent company re-checks (lower risk score = higher priority).

21 Launch Checklist & Post-Build Strategy

Building the project is only half the battle. The 2x-3x salary jump comes from what you do after shipping.

Pre-Launch Checklist

- ☐ App deployed at a real domain with SSL.
- ☐ README with screenshots, architecture diagram, demo video (Loom 2-3 min).
- ☐ Code on GitHub, public repo, clean commit history.
- ☐ Live demo link and GitHub link on resume, LinkedIn, portfolio site.
- ☐ Razorpay in Test Mode — users can upgrade via test cards.
- ☐ Analytics (PostHog) and error tracking (Sentry) wired up.
- ☐ Smoke tests pass: signup, resume upload, app submission, chat, payment upgrade.

Post-Launch Strategy (Next 60 Days)

- Post a LinkedIn launch announcement with screenshots + demo video. Tag relevant hiring managers you admire.
- Write 4 technical blog posts: (1) Playwright anti-bot strategies, (2) LLM structured output with Zod validation, (3) Razorpay webhook idempotency, (4) building the weekly monitor agent.
- Cross-post on dev.to, Medium, and Hashnode.
- Get 10 real users (friends, family, college batchmates job hunting). Gather feedback. Iterate.
- Add 'built this while job hunting' to your cover letters — interviewers ask about it immediately.
- Apply to 50+ jobs across Ahmedabad, Pune, Bangalore, remote. Use the app itself.
- Track every interview: what they asked, what stood out about your project, what fell flat. Iterate your pitch.

YOU'VE GOT THIS — START TODAY

Don't read this entire PDF and feel overwhelmed. Open your terminal right now and run:

```
npx create-next-app@latest job-agent --typescript --tailwind --app
```

That's it. Day 1, Step 1. The rest will follow. Build in public. Push to GitHub daily. Write messy code first, clean it later. Every senior developer started exactly where you are right now.

Four months from today, you'll have a production SaaS, a portfolio that gets interviews, and the skills to legitimately target ₹55-80K+ roles. Start.